ROS2 Learning Journey and practical applications

Mohammad Nawar Amayri

AGENDA

ROS2 Nodes

Publishers and Subscribers

Services

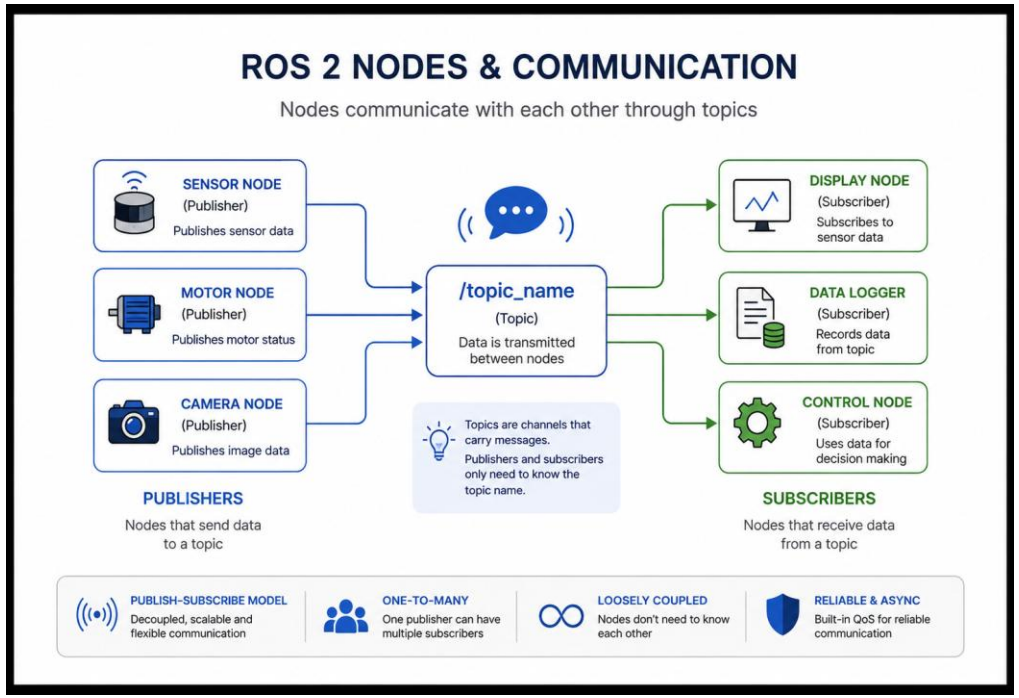
Actions

Comparison Table

Demos of My Tasks

Nodes and Communication

- Nodes are the fundamental units of robotic systems in ROS2.
- Hardware drivers whether considered Sensor or Actuators are examples of ROS2 Nodes. Ex: Lidar, motor, camera, etc..
- Nodes communicate through topics

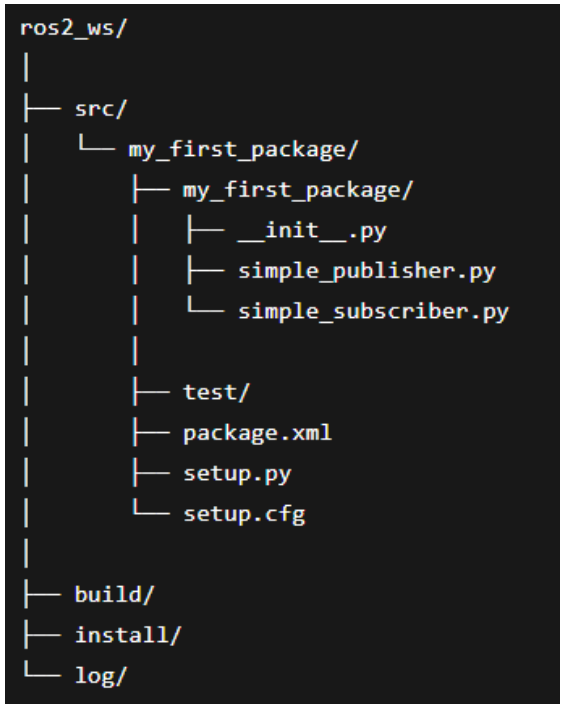


Programming a Node:

```python
import rclpy
from rclpy.node import Node
class MyNode(Node):
    def __init__(self):
        super().__init__('my_node_name')
        # Publisher
        self.publisher_ = self.create_publisher(MessageType,'/topic_name',10)
        # Subscriber
        self.subscription_ = self.create_subscription( MessageType, '/topic_name',
self.callback_function, 10)
        # Timer (optional)
        self.timer_ = self.create_timer( 1.0, self.timer_callback)
        self.get_logger().info('Node Started')
    def callback_function(self, msg):
    """Called whenever a message is received"""
        pass
    def timer_callback(self):
        """Called periodically by timer"""
        pass
def main(args=None):
    rclpy.init(args=args)
    node = MyNode()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    node.destroy_node()
    rclpy.shutdown()
if __name__ == '__main__':
    main()
```

Publishers and Subscribers

- A publisher is a node that sends messages to a topic
- It “broadcasts” the message without caring about :
  - Who is listening
  - The topic being empty
- A Subscriber is a node that listens to the messages from a topic
- It “Receives” from the topic without caring about:
  - Who is sending
  - The topic being empty



Publisher & Subscriber Package Structure

- In the publisher file:
  - We create a node
  - Create a publisher
  - Create a timer (Optional)
  - Create a message
  - Publish the message to the topic

- In the subscriber file:
  - We create a node
  - Subscribe to a topic
  - Receive messages
  - Process Data

Services

- ROS2 Services are a mechanism that follows “Request-Response Pattern”
- Services key characteristics are:
  - One to many: A single server with possibly multiple clients
  - Synchronous: A client would wait for the response
  - Persistent: Services remain available while the program is running until the node shuts down
- Because of these characteristics, Services can be used to change robot movement modes, speeds, and directions

Services Package Structure

- In srv/ we write the input message structure, a separating line “---” and below it we add the response message
- Ex:
- float32 x
- float32 y
- ---
- string name

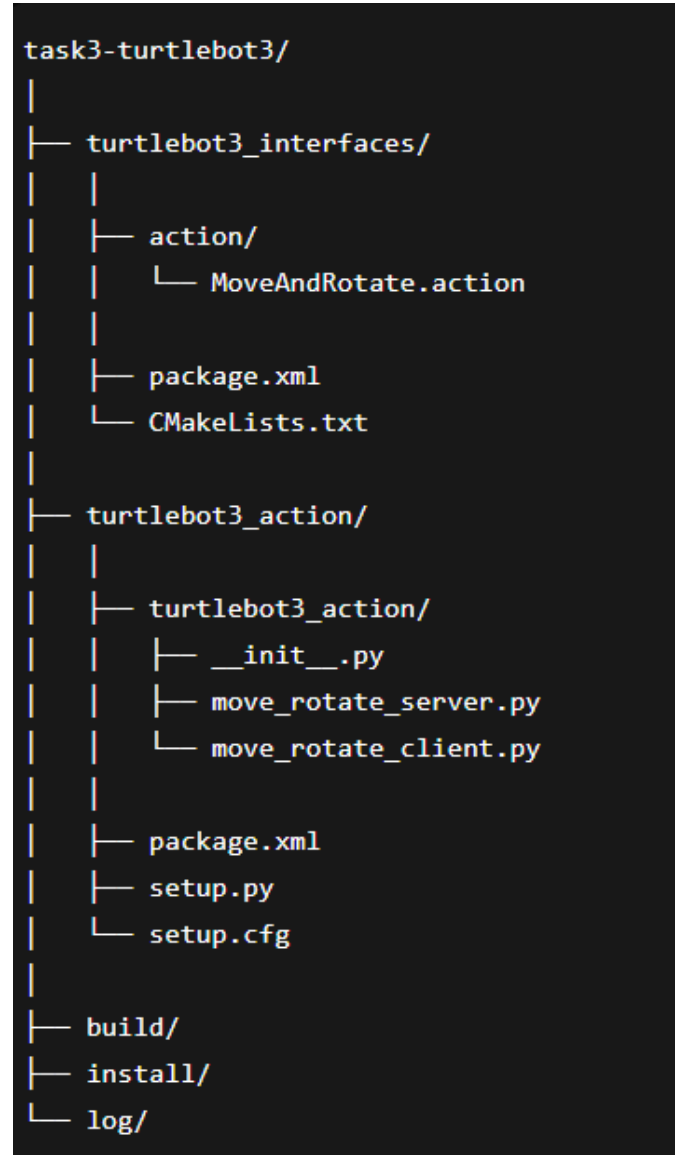
```
interface_package/
│
├── srv/
│   └── ServiceName.srv
│
├── package.xml
└── CMakeLists.txt


service_package/
│
├── service_package/
│   ├── service_server.py
│   └── service_client.py
│
├── package.xml
├── setup.py
└── setup.cfg
```

Actions

- ROS2 Actions are a mechanism that follow “goal-feedback-result”
- Actions key characteristics are:
  - Feedback: The Server provides back Real-time updates on the task progress
  - Asynchronous: The client continues while the goal is being processed without being blocked
  - Cancelable: Clients can cancel the goal before completion
- Because of these characteristics, Actions are best suited for long-running tasks where progress updates and goal cancellation may be required.

Actions Package Structure

- In actions/ we write the input the goal, a separating line “---” and below it we add the result, another separating line “---” and below it we add the feedback
- Ex:
- float32 theta
- ---
- float32 delta
- ---
- float32 remaining



Code Structures inside Server and Client Files

Server File

```python
import rclpy
from rclpy.node import Node
from package_name.srv import ServiceName
class ServiceServer(Node):
    def __init__(self):
        super().__init__('service_server')
        self.service = self.create_service(
            ServiceName, 'service_name', self.callback)
    def callback(self, request, response):
        # Read request data
        request.field_1
        request.field_2
        # Process request
        # Fill response
        response.success = True
        return response
def main():
    rclpy.init()
    node = ServiceServer()
    rclpy.spin(node)
    rclpy.shutdown()
```

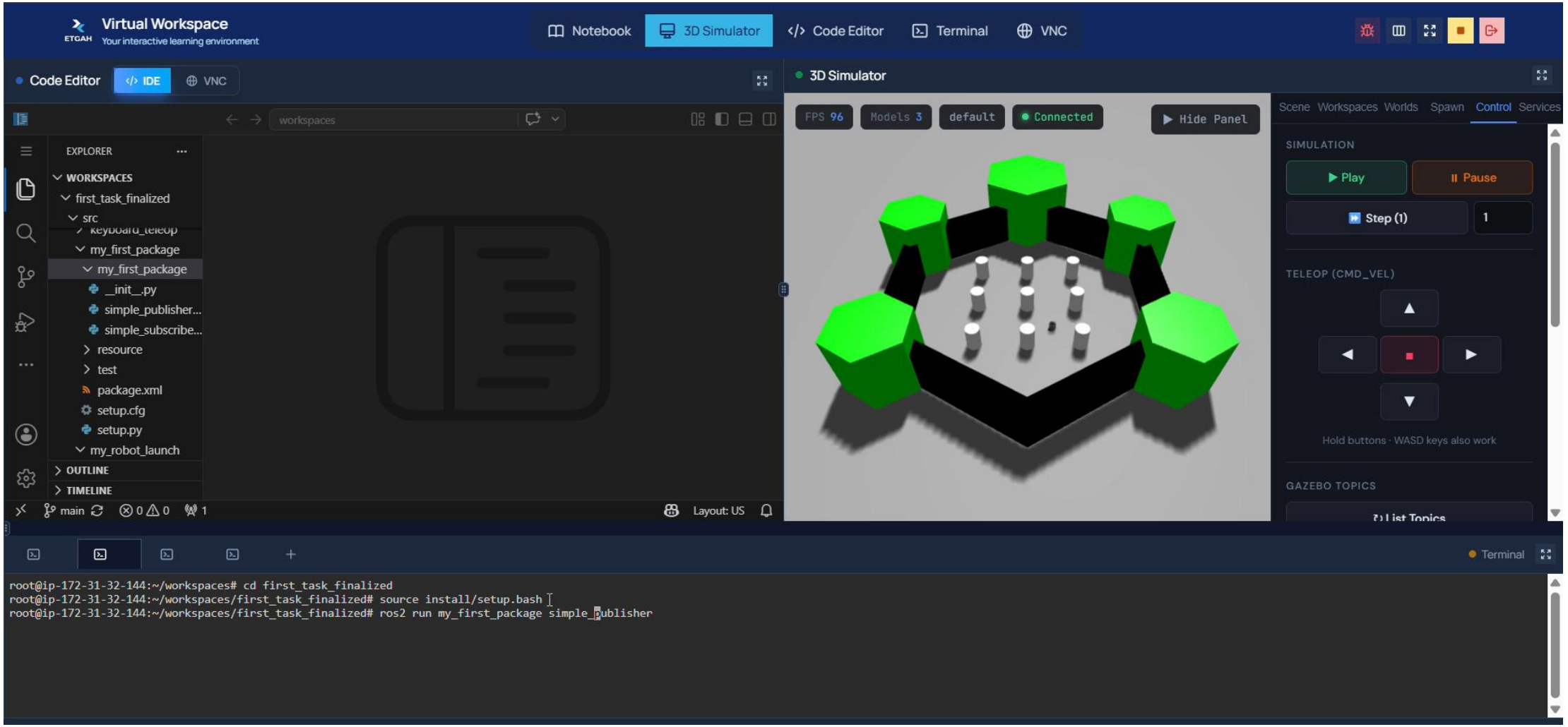
Client File

```python
import rclpy
from rclpy.node import Node
from package_name.srv import ServiceName
class ServiceClient(Node):
    def __init__(self):
        super().__init__('service_client')
        self.client = self.create_client(
            ServiceName,'service_name')
        while not self.client.wait_for_service(
            timeout_sec=1.0):
            self.get_logger().info(
                'Waiting for service...')
    def send_request(self):
        request = ServiceName.Request()
        request.field_1 = value_1
        request.field_2 = value_2
        future = self.client.call_async(request)
        future.add_done_callback(
            self.response_callback)
    def response_callback(self, future):
        result = future.result()
        print(result)
def main():
    rclpy.init()
    node = ServiceClient()
    node.send_request()
    rclpy.spin(node)
    rclpy.shutdown()
```

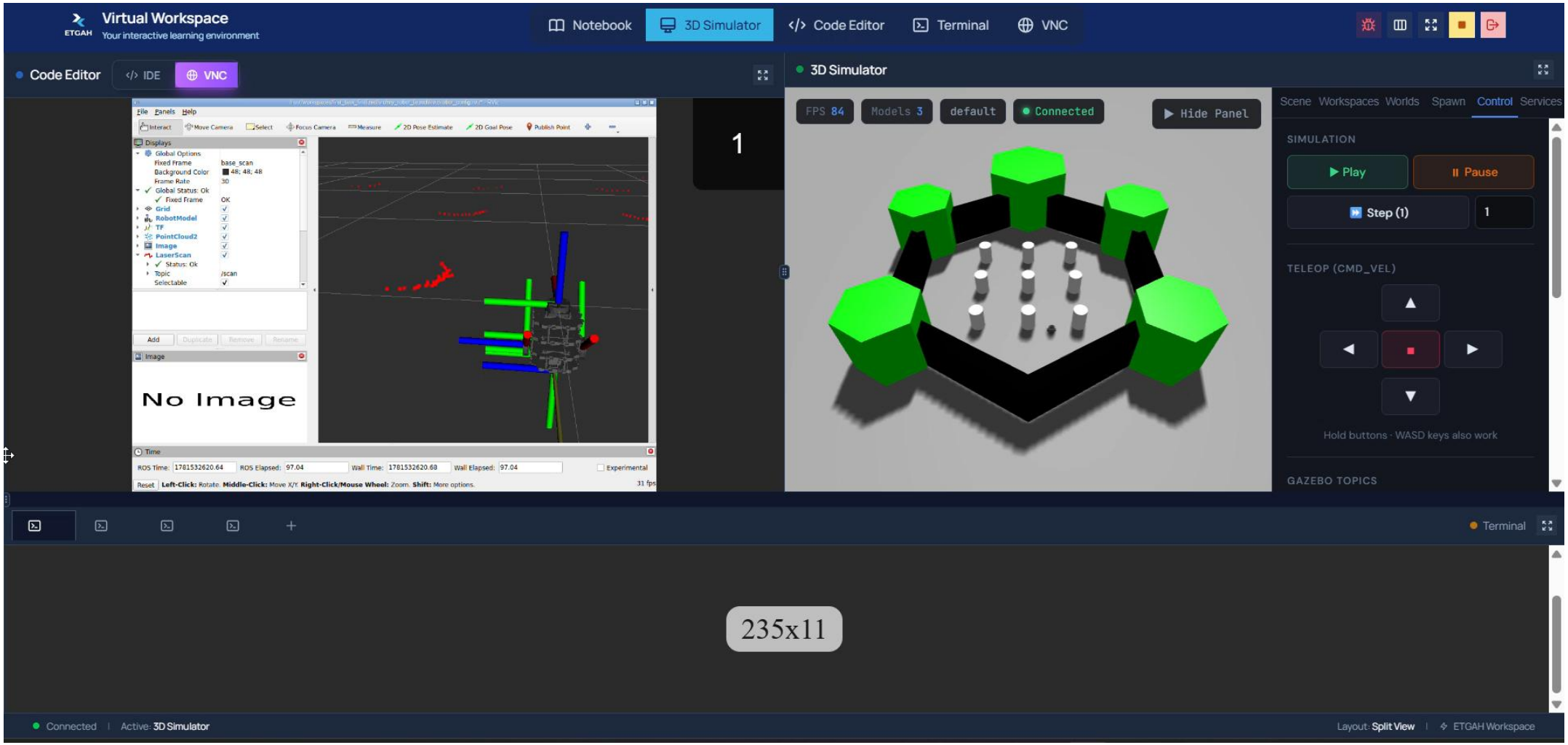
Feature	Topics	Services	Actions
Communication type	Publisher/Subscriber	Request/Response	Goal/Feedback/Result
Data Flow	Continuous stream of messages	One time request and response	Long-running tasks
Best Use Cases	Sensor data, robot control	Configuration changes and parameter updates while running	Navigation, autonomous missions
Cancellation Support	No	No	Yes
Response Expected	No	Immediate Response	Final result after completion and constant feedback

Tasks Demos

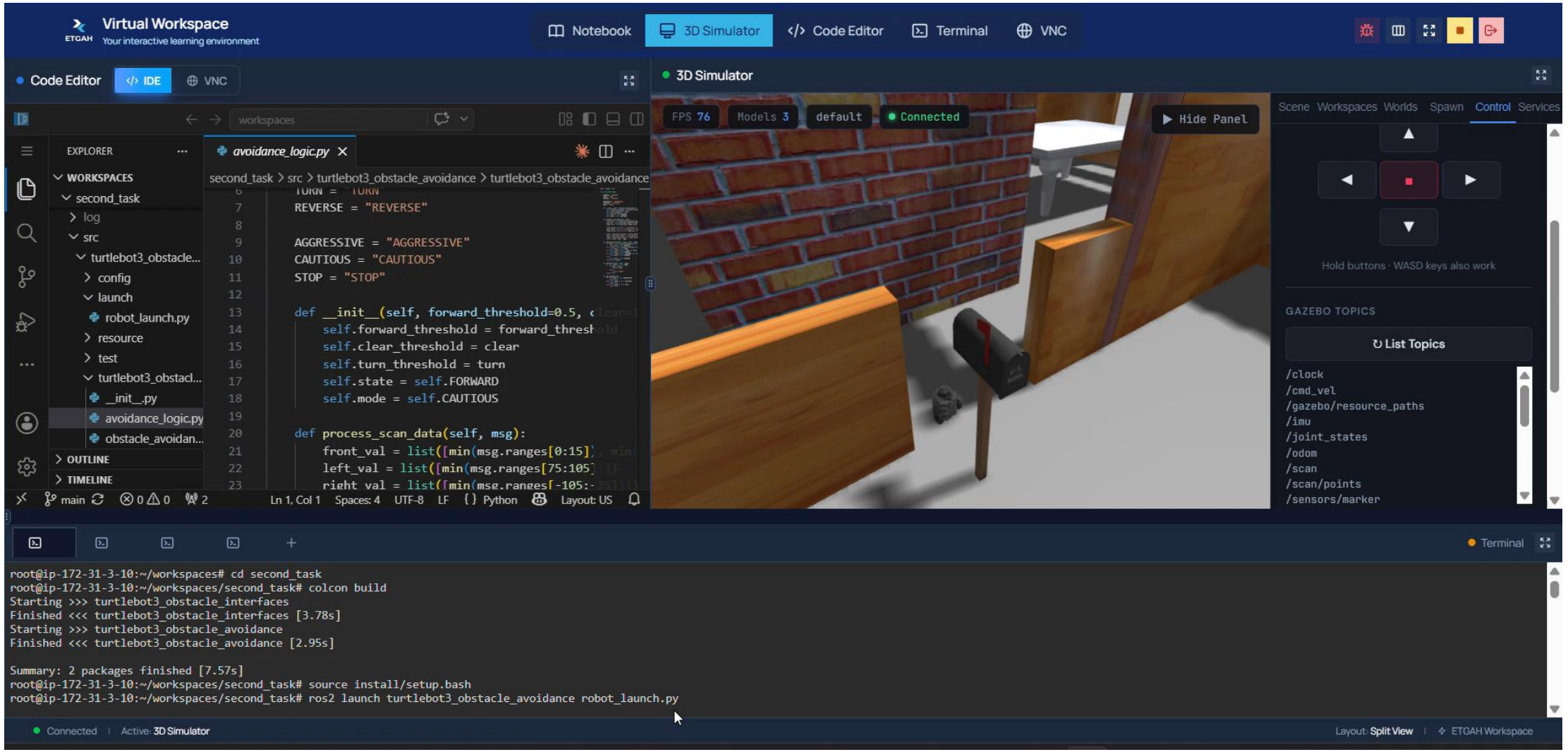
Task1-Publisher and Subscriber Node



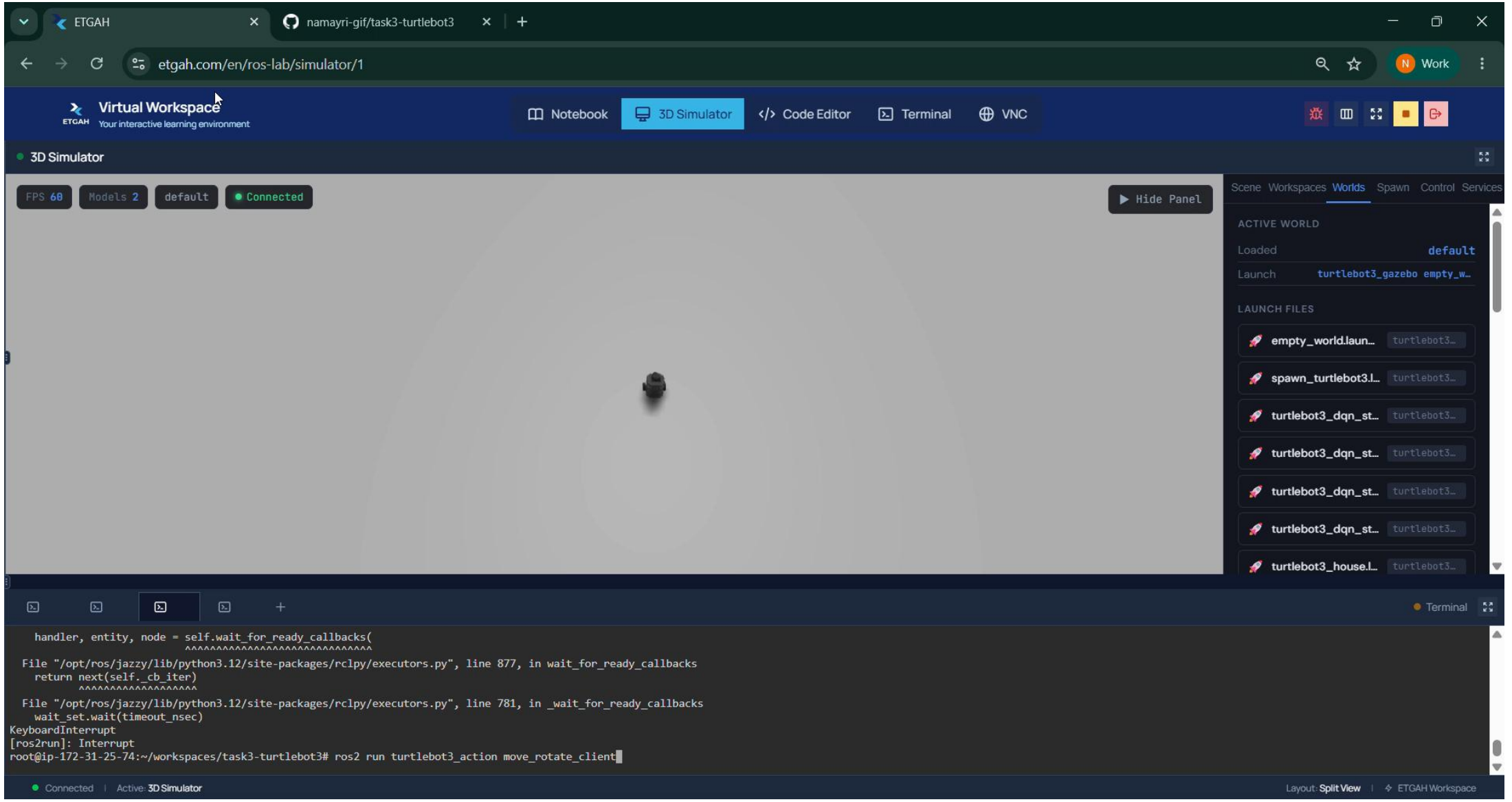
Task 1-Rviz Launch file



Task 2-Obstacle Avoidance ROS2 Services



Task 3: Turtlebot3 Action Task



References

ETGAH Material Documents

Claude

Google Gemini